

수신호 기반 공정 기계 안내 시뮬레이터

프로젝트 구조 문서 — 웹캠 수신호 인식 → ROS 2 → Gazebo 기계 제어

2026-07-09 기준 · 저장소: Mocap-Sample (인식) / Mocap-Sim (시뮬레이션)

1. 프로젝트 개요

웹캠 영상에서 작업자의 **산업 수신호(hand signal)**를 실시간 인식하고, 분류 결과를 ROS 2 토픽으로 전달하여 Gazebo 시뮬레이션의 공정 기계가 대응(정지·감속·접근·선회)하는 시스템. [2026-07-07 end-to-end 실증 완료](#)

- Mocap-Sample** (macOS/Windows) — 인식 담당: 데이터 수집 → 학습 → 실시간 추론 → 결과 전달
- Mocap-Sim** (Ubuntu 24.04) — 기계 담당: ROS 2 수신 → 안전 규칙 → Gazebo 기계 구동

2. 시스템 아키텍처

```
[Mocap-Sample - macOS/Windows]
  카메라 입력 (OpenCV, 30 FPS)
    |
  손+포즈 랜드마크 추출 (MediaPipe HandLandmarker + PoseLandmarker)
    |   양손 21점×2 + 상체 8관절 = 50포인트 × xyz = 150차원/프레임
    |
  슬라이딩 윈도우 버퍼 (최근 30프레임)
    |
  LSTM 분류 (2층, hidden 64) → 클래스 확률
    |
  안정화: 확률 EMA 평활 → 신뢰도 ≥0.8 + 연속 5회 → 확정
    |   (해제는 1초 유예 / 사람 미감지·stop은 즉시)
    |
  rosbridge 웹소켓 발행 —————→ JSON {"signal","confidence","timestamp"}
                                     ▼
[Mocap-Sim - Ubuntu 24.04 VM]
  rosbridge 서버(:9090) → ROS 2 토픽 /hand_signal
    |
  signal_listener 노드 (신호→속도 매핑, 하트비트 2초 감시)
    |
  /cmd_vel → ros_gz_bridge → Gazebo 공정 기계 (차동구동)
```

3. 저장소 구조

Mocap-Sample (인식)

```
Mocap-Sample/
├── README.md           # 사용법·태스크 안내
├── pyproject.toml      # 의존성 버전 고정 (Python 3.12)
├── .vscode/           # 셋업·실행 태스크 (크로스 플랫폼)
├── doc/
│   └── AGENT.md       # 코딩 에이전트 안내 (셋업·제약·현재 상태)
```

```

├── design.md          # 아키텍처·설계 결정·ODD 약조건 대응표
├── ml-flow.md         # 데이터→학습→예측 방법론, 체크리스트
├── TODO.md           # 진행 기록·할 일 (단일 기준점)
├── physical-ai-notes.md # 센서 필터링·안전 제어 기법 노트
├── asset/<라벨>/*.npz  # 학습 데이터 (30, 150) - 폴더명 = 클래스
├── models/
│   ├── *.task        # MediaPipe 모델 (자동 다운로드)
│   └── sign_classifier.pt # 학습된 LSTM + 라벨 목록 + 설정
├── src/
│   ├── capture/
│   │   ├── extractor.py      # ★ 공용 특징 추출기 (손+포즈 150차원)
│   │   └── landmark_viewer.py # 시각화 도구 + 손 이탈 경고
│   ├── dataset/
│   │   ├── collect.py       # 자동 연속 수집 (버튼 UI, 품질 필터)
│   │   └── import_jester.py  # 외부 데이터셋 변환 (실험용 유지)
│   ├── training/
│   │   ├── train.py         # LSTM 학습·평가·저장 (클래스 동적 구성)
│   │   ├── augment.py       # 약조건 증강 (도메인 랜덤화)
│   │   └── stress_test.py    # 조건별 강건성 측정
│   └── inference/
│       ├── predict.py       # 실시간 추론 + 안정화 + 안전 가드 + UI
│       └── publisher.py     # rosbridge/UDP 결과 전달 (하트비트)

```

Mocap-Sim (시뮬레이션)

```

Mocap-Sim/
├── README.md          # Ubuntu 설치·실행 가이드 (VM/네이티브 경로)
├── src/signal_bridge/  # ROS 2 패키지 (ament_python)
│   ├── signal_bridge/signal_listener.py # /hand_signal → /cmd_vel + 안전 규칙
│   ├── launch/sim.launch.py             # Gazebo+브리지+rosbridge+노드 일괄 실행
│   └── worlds/factory.sdf                # 공정 기계(차동구동) 월드
├── tools/
│   ├── gz_signal_adapter.py # Mac 단독 테스트용 UDP→gz 어댑터
│   └── run_demo_mac.py      # Mac 데모 통합 런처

```

설계 원칙: 수집·변환·추론이 모두 `extractor.py` 한 모듈을 사용해 학습-추론 특징 형식 불일치를 원천 차단. 클래스 목록은 코드에 하드코딩하지 않고 `asset/` 폴더 스캔 → 모델 파일에 라벨 동봉 → 추론이 로드.

4. 수신호 클래스와 기계 행동

라벨	동작	기계 행동	데이터
stop	손바닥 정면으로 내밀기	즉시 정지	30개
slow	손바닥 아래로 눌러 내리기	감속 (0.1 m/s)	30개
come	손짓해 부르기	접근 (0.4 m/s)	30개
back	손등으로 밀어내기	후진	미수집
left_go / right_go	좌/우로 팔 젖기	선회	각 30개
idle	평상시 동작	직전 상태 유지	30개
unknown	(인식 불가 상태)	정지 — 안전 기본값	—

현재 모델: 6클래스, test 정확도 77.8% (라벨당 30개 기준). 목표: 라벨당 100개 보강 후 90%+.

5. 머신러닝 학습 방법

5-1. 데이터 수집과 표현

영상(픽셀)을 직접 학습하지 않고 **MediaPipe 랜드마크 좌표를 특징으로 사용**한다. 배경·조명·옷차림 변화에 구조적으로 강하고, 데이터가 작아(샘플당 18KB) 소량 수집으로도 학습이 가능해지는 것이 핵심 이유다.

카메라 프레임 → HandLandmarker + PoseLandmarker (extractor.py)
→ [왼손 21점×xyz 63 | 오른손 63 | 상체 8관절×xyz 24] = 150차원/프레임
→ 연속 30프레임(약 1초) = 샘플 1개, shape (30, 150) → asset/<라벨>/*.npy

- **정규화**: 좌표는 화면 기준 0~1로 이미 정규화. 손목 상대좌표화는 팔의 이동 정보(모션)를 없애므로 쓰지 않고 원시 좌표를 유지
- **부재 처리**: 손/포즈 미감지 프레임은 해당 구간 0 — "안 보임" 자체가 정보
- **수집 도구**: collect.py가 자동 연속 수집(카운트다운→녹화→저장 반복) + 품질 필터(손 감지율 50% 미만 폐기, idle은 예외)로 라벨링 오류를 차단

5-2. 모델과 학습 절차 (train.py)

항목	내용
모델	LSTM 2층 (hidden 64) → FC → 클래스 확률. 시퀀스 분류의 최소 베이스라인 — "가장 단순한 모델로 시작, 부족할 때만 복잡도 상승" 원칙 (다음 후보: 1D CNN, ST-GCN)
클래스 구성	asset/ 라벨 폴더 스캔으로 동적 결정 (하드코딩 금지), 라벨 목록은 모델 파일에 동봉
분할	train 70 / val 15 / test 15, 클래스 비율 유지(충화), 고정 시드

최적화	cross-entropy 손실 + Adam, val 정확도 기준 early stopping (30 epoch 무개선 시 중단, 최고 시점 가중치 복원) — 소량 데이터 과적합 방지
증강	학습 배치에만 실시간 적용하는 특징 수준 도메인 랜덤화 (augment.py): 좌표 노이즈·손 프레임 드롭아웃(가림)·시간 신축(속도 차)·이동/스케일(위치·거리)·렌즈 왜곡. 부재 지점(0)은 보존, val/test 미적용
평가	test 정확도 + 혼동 행렬 + 클래스별 정밀도/재현율. 안전 관점 최우선 지표는 stop 재현율 (놓치면 사고, 오탐은 정지 방향이라 허용)
저장	<code>models/sign_classifier.pt</code> — 가중치+라벨 목록+설정 일체 (추론이 별도 설정 없이 로드)

5-3. 검증된 교훈과 개선 루프

- **외부 데이터셋 실패 (Jester 실험)**: 대형 제스처 데이터셋으로 학습 시 내부 test 88~91%였으나 실제 웹캠에서 크게 빗나감 — 도메인 갭(동작 방식·속도) + 셀피 뷰 좌우 반전. → **배포 환경과 동일 파이프라인으로 자체 수집한 데이터만 사용**으로 전환
- **혼동 쌍은 데이터가 아니라 동작 정의 문제**: '멈춰'와 '오지마'가 혼동 행렬에서 섞여 stop 하나로 통합 — 모델 튜닝보다 클래스 재설계가 먼저
- **개선 루프**: 실사용 오분류 관찰 → 그 라벨/상황만 조준 수집(같은 라벨 재실행 시 이어서 축적) → 재학습 → 재평가. 모델 구조 변경보다 데이터 보강이 항상 우선
- **강건성 정량화**: stress_test.py가 악조건(노이즈·가림·왜곡 등)별 test 정확도 하락을 측정 — 현재 전 조건 2%p 이내

5-4. ML 라이브러리 구성과 선택 이유

학습·예측의 수학(레이어 연산, 역전파, 최적화)은 전부 라이브러리가 담당하고, 직접 작성한 코드는 데이터 적재·학습 루프 순서·저장 형식 등 접착 코드다.

역할	라이브러리	담당
학습·예측 엔진	PyTorch 2.12.1	LSTM 레이어, cross-entropy 손실, Adam 최적화, 역전파, 추론
분할·평가	scikit-learn 1.9.0	총화 train/val/test 분할, 혼동 행렬, 정밀도/재현율
특징 추출	MediaPipe 0.10.35	손·포즈 랜드마크 검출 (구글이 사전학습한 딥러닝 모델)
수치 연산	NumPy 2.5.0	.npy 데이터 입출력, 증강 변환

검토했던 대안과 미채택 이유:

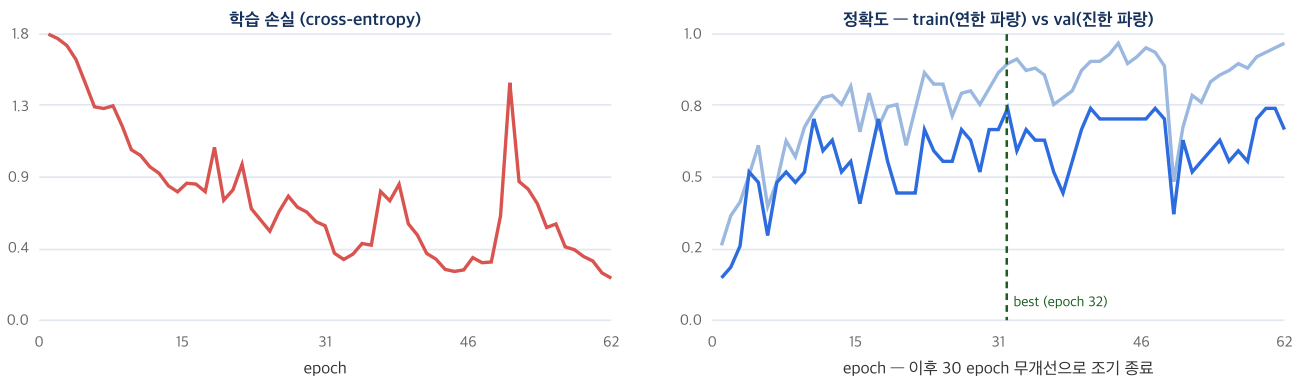
- **MediaPipe Gesture Recognizer** (기성 커스텀 제스처 학습) — 정지 이미지 1장 기준의 **정적** 제스처 전용. 우리 수신호는 30프레임 **동작(모션)**이라 시퀀스 모델이 필수 → 사용 불가
- **Keras / TensorFlow** — 기술적으로 동등하게 가능 (LSTM 지원 동일). PyTorch를 택한 것은 우열이 아니라 선택의 문제이며, 현재 파이프라인이 검증된 상태라 교체 시 학습·추론 코드 재작성 비용만 있고 정확도 이득은 없음
- **YOLO + LSTM** (크레인 수신호 논문들의 구성) — YOLO는 공간 탐지(어디에 뭐가 있나), LSTM은 시간 분류로 역할이 달라 대체 관계가 아님. 우리 구조는 공간 담당만 MediaPipe로 바꾼 같은 패턴이며, 바운딩 박스보다 관절 좌표가 수신호에 정보량이 큼. YOLO가 필요해지는 건 원거리 CCTV·다중 인원에서 "신호수 찾기"가 먼저일 때
- **이미지(픽셀) 직접 학습** (3D-CNN 등) — 배경·조명이 입력에 들어와 라벨당 수천 장이 필요하고 GPU 필수, 랜드마크의 배경 불변 장점을 상실. 렌즈 왜곡의 기하 효과는 특징 수준 증강으로 동등하게 커버되며, 실물 왜곡 렌즈는 학습이 아니라

캘리브레이션(`cv2.undistort`) 전처리가 정석

- **scikit-learn 기성 분류기** (SVM, RandomForest) — 시간 순서를 모델링하지 못해 (30, 150) 시퀀스를 4,500차원으로 펴야 함 → 동적 제스처에 구조적으로 불리
- **PyTorch Lightning** (학습 루프 래퍼) — 학습 코드가 이미 소규모라 이득 대비 의존성 증가만 있음

5-5. 학습 과정 시각화 — 실제 학습 곡선과 혼동 행렬

아래는 이 저장소의 실제 데이터(6클래스 × 30개)로 학습을 1회 실행해 기록한 결과다.



- **손실 곡선**: 경사하강이 가중치를 조정하며 $-\log p_{\text{정답}}$ 이 감소하는 과정. 매 epoch 증강이 다른 변형을 만들어 곡선이 매끈하지 않고 출렁인다 — 정상
- **train vs val 간격이 과적합의 신호**: train(연한 파랑)이 val(진한 파랑)보다 크게 앞서기 시작하면 암기 중이라는 뜻. best 지점(초록 점선, epoch 32, val 74%)의 가중치만 저장하고, 이후 30 epoch 무개선으로 조기 종료해 과적합 구간을 잘라낸다

혼동 행렬 (test 27개) — 행=실제, 열=예측, 대각선=정답

	come	idle	left_go	right_go	slow	stop
come	3	0	0	0	1	0
idle	0	2	0	2	0	0
left_go	0	0	4	1	0	0
right_go	1	0	0	4	0	0
slow	0	0	0	0	5	0
stop	0	0	0	0	1	3

이 실행의 test 정확도 77.8%. 혼동 행렬의 대각선이 정답, 대각선 밖(붉은 칸)이 오분류 쌍이다 — 어떤 신호끼리 헷갈리는지 가 보이므로 "오분류 조준 수집"의 근거 자료가 된다.

6. 학습·추론에 쓰인 수학 (선형대수)

6-1. 데이터의 형태 — 벡터와 행렬

프레임 1장의 특징: $x_t \in \mathbb{R}^{150}$ (150차원 벡터)
샘플(동작) 1개: $X = [x_1; \dots; x_{30}] \in \mathbb{R}^{30 \times 150}$ (30×150 행렬)
데이터셋 전체: $(N, 30, 150)$ 3차원 텐서 — N = 샘플 수

수집·증강·학습·추론 전 단계가 이 벡터/행렬을 주고받는 선형대수 연산이다.

6-2. LSTM 순전파 — 행렬 곱과 게이트

매 프레임 t 마다 아래 여섯 식을 계산한다 (σ = 시그모이드, $\odot$ = 성분별 곱):

$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$ 입력 게이트 — 새 정보를 얼마나 받을까
 $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$ 망각 게이트 — 과거를 얼마나 지울까
 $g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g)$ 후보 상태 — 새로 기억할 내용
 $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$ 출력 게이트 — 기억을 얼마나 내보낼까
 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ 셀 상태 (장기 기억)
 $h_t = o_t \odot \tanh(c_t)$ 은닉 상태 (프레임 t 까지의 요약)

핵심 연산은 전부 **행렬×벡터 곱**: $W \in \mathbb{R}^{64 \times 150}$ (입력→은닉), $U \in \mathbb{R}^{64 \times 64}$ (이전 은닉→은닉). 모델 파라미터 약 8.9만 개의 대부분이 이 행렬들의 원소다. 30프레임을 순서대로 통과시킨 마지막 $h_{30} \in \mathbb{R}^{64}$ 가 "동작 전체를 요약한 벡터"가 된다 — 시간 정보를 기억하는 게이트 구조가 LSTM이 시퀀스 분류에 쓰이는 이유.

6-3. 분류층과 소프트맥스 — 요약 벡터를 확률로

로짓(점수): $z = W_c h_{30} + b_c \in \mathbb{R}^K$ (K = 클래스 수, $W_c \in \mathbb{R}^{K \times 64}$)
소프트맥스: $p_i = \exp(z_i) / \sum_j \exp(z_j)$ (합이 1인 확률 분포로 변환)

추론 UI의 클래스별 확률 막대와 신뢰도 임계값(0.8) 판정이 이 p 를 그대로 사용한다.

6-4. 학습 — 손실 함수와 경사하강

손실(cross-entropy): $L = -\log p_{\text{정답}}$ 정답 확률이 1이면 0, 낮을수록 커짐
경사하강(Adam): $\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$ 기울기 반대 방향으로 파라미터 미세 조정

- $\partial L / \partial \theta$ (기울기)는 역전파가 연쇄법칙(chain rule)으로 계산 — PyTorch autograd 담당
- Adam은 기울기의 평균·분산 이력으로 학습률 η 를 파라미터별 자동 조절
- 과적합 방지: val 정확도가 최고인 시점의 가중치를 저장해두고 30 epoch 무개선 시 중단 후 복원 (early stopping)

6-5. 추론 안정화 — 지수이동평균(EMA)

$$s_t = \alpha \cdot p_t + (1-\alpha) \cdot s_{t-1} \quad (\alpha = 0.4)$$

확률 벡터의 시간 방향 가중 평균. 전개하면 과거 값에 $(1-\alpha)^n$ 으로 지수 감쇠하는 가중치가 걸려, 1프레임 스파이크는 눌리고 지속되는 변화만 통과한다 — 확정/해제 판정은 날값 p 가 아니라 평활값 s 로 한다.

6-6. 증강의 기하 변환

평행이동: $x' = x + t$ (작업자 위치 변화)
 스케일: $x' = c + s \cdot (x - c)$ (카메라 거리 변화, 중심 c 기준 상수배)
 렌즈 왜곡: $r^2 = dx^2 + dy^2$ (중심으로부터의 거리 제곱)
 $x' = c + (1 + k \cdot \min(r^2, 0.5)) \cdot (x - c)$ (반경에 비례한 방사형 변형)
 시간 신축: 30프레임 $\leftrightarrow$ 24~36프레임 선형 보간 (동작 속도 변화)

모두 좌표 벡터에 대한 아핀/방사 변환 — 이미지 없이 특징 수준에서 같은 기하 효과를 낸다.

6-7. 평가 지표

정확도 = 맞춘 샘플 / 전체 정밀도 = $TP / (TP+FP)$ "그 신호라고 한 것 중 진짜 비율"
 재현율 = $TP / (TP+FN)$ "진짜 그 신호 중 잡아낸 비율"

안전 최우선 지표는 stop 재현율 — 놓친 stop(FN)이 사고로 직결되기 때문. 오탐(FP)은 기계가 정지하는 방향이라 허용 가능.

7. 안전 설계 (계층 방어)

계층	방어선	동작
추론 — 확신도	신뢰도 임계값 0.8 + 연속 5회 일치	불확실하면 확정하지 않음
추론 — 노이즈	확률 EMA 평활 + 해제 1초 유예	스파이크 1프레임에 기계가 서지 않음
추론 — 존재	포즈 감지율 30% 미만 가드	사람이 없으면 즉시 unknown
전달	1초 주기 하트비트	통신 상태를 기계가 항상 인지
기계	unknown→정지 / 하트비트 2초 초과→무조건 정지	인식기가 죽어도 기계는 멈춤
학습	악조건 증강 (노이즈·가림·왜곡 등 5종)	좌표 오염에 강건 (전 조건 하락 2%p 이내)

악조건 분류 전체(운영 범위 안=인식 유지 / 밖=안전 정지)는 design.md의 ODD 대응표가 관리.

8. 기술 스택

구성	기술
랜드마크 추출	MediaPipe HandLandmarker + PoseLandmarker (Tasks API)
분류 모델	PyTorch LSTM 2층 (파라미터 약 8.9만 개, CPU 학습 수 분)
영상 처리	OpenCV (카메라 백엔드 OS별 분기)
결과 전달	rosbridge 웹소켓 (+UDP 디버깅 백엔드)
시뮬레이션	ROS 2 Jazzy + Gazebo Harmonic, Ubuntu 24.04 (UTM VM, CPU 렌더링)
환경	Python 3.12, 의존성 버전 고정, Windows/macOS 크로스 플랫폼

9. 실행 방법 (VSCode 태스크)

태스크	용도
프로젝트 셋업 (Cmd/Ctrl+Shift+B)	venv 생성 → 설치 → 검증 일괄
실행: 랜드마크 시각화	카메라·프레이밍 점검
실행: 데이터 수집	라벨 입력 → 자동 연속 수집
실행: 모델 학습	학습 + 혼동 행렬 평가 + 저장
실행: 실시간 추론	웹캠 인식 (확률 패널 UI)
시물 데모 (Mac)	Gazebo 네이티브 + 어댑터 + 인식기 일괄 실행

ROS 2 연동: `predict --publish rosbridge --rosbridge-url ws://<시물IP>:9090`

10. 진행 상태와 로드맵

- **완료:** 전체 파이프라인(수집→학습→추론→전달→기계 반응) 실증, 악조건 증강·강건성 측정, 안정화 고도화(EMA+유예), 문서 체계 5종
- **다음:** 데이터 보강(라벨당 100개, back 추가) → 재학습 90%+ → 악조건 실측 테스트(저조도·역광·장갑)
- **백로그:** 시야 자가 진단(/vision_health), 구역 기반 점진 감속, APF 경로 계획, 다중 사용자 일반화 — 상세는 doc/TODO.md